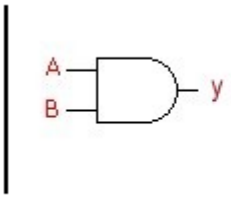
Overview of Verilog VHDL

- If you can express working of a digital circuit and visualize the flow of data inside a IC, then learning any HDL or Hardware Description Language is very easy.
- Let us start with an AND gate. Here is the truth table:



A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

- **The Code**

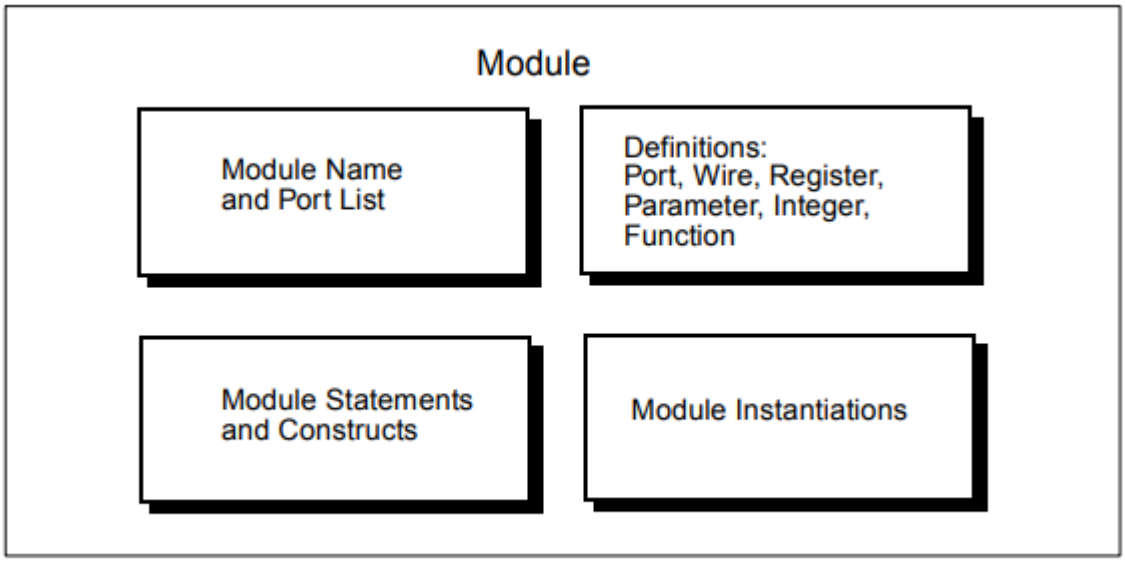
```
/* A simple AND gate
File: and.v                    */

module andgate (a, b, y);
input a, b;
output y;

assign y = a & b;

endmodule
```

module module_name (port_list);
declarations:
port declaration (input, output, inout, …)
data type declaration (reg, wire, parameter, …)
task and function declaration
statements:
initial block — Behavioral
always block — Behavioral
module instantiation — Structural
gate instantiation — Structural
UDP instantiation — Structural
continuous assignment — Data-flow
endmodule

- In verilog, one circuit is represented by set of "modules". We can consider a module as a black box.

- In the beginning of a module, we have to declare all ports as input output or inout. By default, ports will have one pin or one bit.
- Using 'assign' statement, we connected inputs and outputs via AND gate. A will be ANDed with B and will be connected to Y.
- Here, we are not going to store the values, and hence we did not declare any registers. By default, all the ports will be considered as wires. If we want the output to be latched, we have to declare it using the keyword 'reg'.

Structural Parts



Module Statements and Constructs

• parameter declarations

```
parameter TRUE=1, FALSE=0;

parameter [1:0] S0=3, S1=1, S2=0, S3=2;
```

• wire, wand, wor, tri, supply0, and supply1 declarations

```
wire a;
wire [2:0] b;
supply0 gnd;
supply1 power;
```

• reg declarations

```
reg x; //single bit
reg a,b,c; //3 1-bit quantities
reg [7:0] q; //an 8-bit vector
```

PORT DECLARATIONS

• input declarations

```
input a;
input [2:0] b;
```

• output declarations

```
output a;
output [2:0] b;
reg [2:0] b;
```

• inout declarations

```
inout a;
inout [2:0] b;
```

• Continuous assignments

• Module instantiations

```
module SEQ(BUS0,BUS1,OUT); //description of module
SEQ input BUS0, BUS1;
 output OUT; ...
 endmodule

module top( D0, D1, D2, D3, OUT0, OUT1 );
 input D0, D1, D2, D3;
 output OUT0, OUT1;
SEQ SEQ_1(D0,D1,OUT0), //instantiations of module
SEQ SEQ_2(.OUT(OUT1), .BUS1(D3), .BUS0(D2));
endmodule
```

• Gate instantiations

• Function definitions

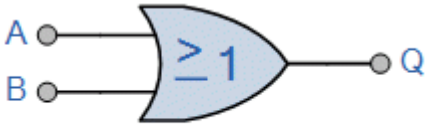
• always blocks

• task statements

OPERATORS

Arithmetic Operators	+ – * / %	Arithmetic Modules
Relational Operators	> >= < <=	Relational
Equality Operators	== !=	Logical equality Logical inequality
Logical Operators	! && ||	Logical NOT Logical AND Logical OR
Bitwise Operators	~ & | ^ ^~ ~^	Bitwise NOT Bitwise AND Bitwise OR Bitwise XOR Bitwise XNOR
Reduction Operators	& | ~& ~| ^ ~^ ^~	Reduction AND Reduction OR Reduction NAND Reduction NOR Reduction XOR Reduction XNOR

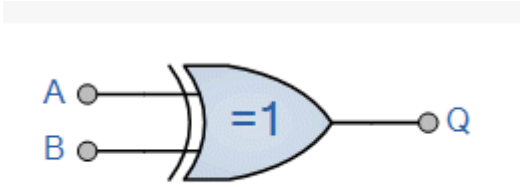
OR Gate Code



B	A	OUT
0	0	0
0	1	1
1	0	1
1	1	1

```
module orgate  ( a, b , y)
   input a, b;
  output y;

assign y= a | b;

endmodule
```

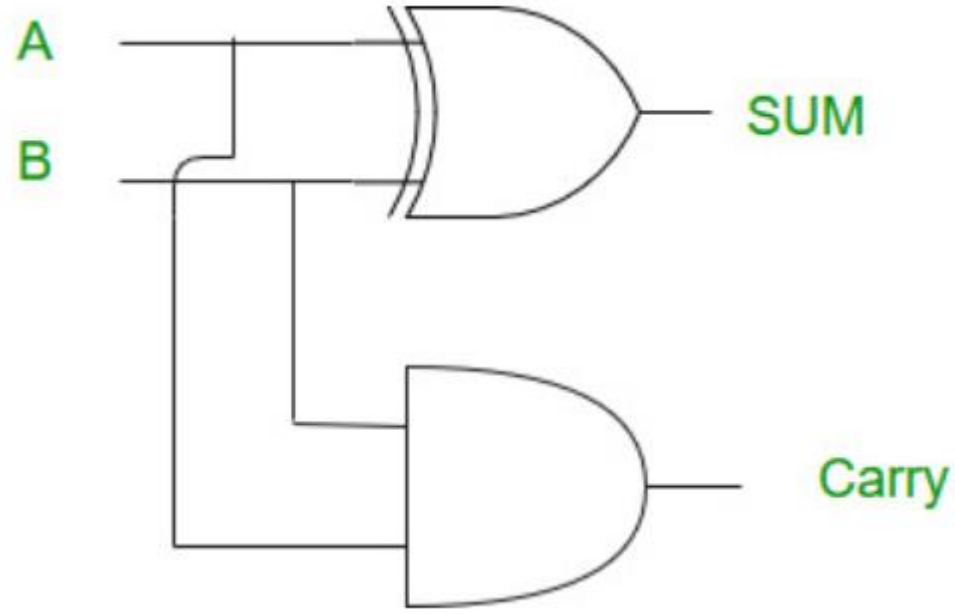
XOR Gate Code



B	A	Q
0	0	0
0	1	1
1	0	1
1	1	0

```
module xorgate  ( a, b , y)
   input a, b;
  output y;

assign y= a ^ b;

endmodule
```

HALF ADDER



A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

```
module half_adder
  (
   i_bit1,
   i_bit2,
   o_sum,
   o_carry
   );

  input  i_bit1;
  input  i_bit2;
  output o_sum;
  output o_carry;

  assign o_sum   = i_bit1 ^ i_bit2;  // bitwise xor
  assign o_carry = i_bit1 & i_bit2;  // bitwise and

endmodule
```

FULL ADDER

Inputs			Outputs	
A	B	C – IN	Sum	C - Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1